MICHAEL ANGELO L. ACUÑA

AI Automation Specialist | Virtual Assistant | Web Developer

Nueva Ecija, Philippines | +63 953 789 0105 | angeloacuna84@gmail.com | linkedin.com/in/angelo-acuna

PROFESSIONAL SUMMARY

AI Automation Specialist, Virtual Assistant, and Web Developer with more than two years of experience in administrative support, customer service, data management, and digital operations. Built and deployed six end-to-end business automation systems using n8n, Next.js, Supabase, Google Workspace, Slack, and AI models. Skilled at turning repetitive processes into practical workflows for recruitment, lead management, customer support, proposal generation, content creation, and invoice processing

WORK EXPERIENCE

Virtual Assistant, Uncle Pinoy Lechons UK **Jan 2024 - April 2026**

- Managed customer inquiries through Facebook Messenger, Instagram, and Outlook
- Assisted in order coordination and delivery scheduling using Shopify
- Maintained accurate customer records and provided timely customer support
- Engaged with customers through comments and page interactions to help increase online engagement and customer response

Data Entry Virtual Assistant, European Client (Remote) **June 2025 - Jan 2026**

- Monitored and updated live data entries with high accuracy and attention to detail
- Maintained spreadsheets and system records using Google Sheets and Excel
- Coordinated with team members to ensure smooth workflow and timely reporting

Front-End Developer Intern, Affirm Technology **Jan 2026 - April 2026**

- Assisted in developing and improving web-based system interfaces using PHP, JavaScript, and Laravel
- Converted design references into responsive and user-friendly web pages

Virtual Assistant Intern - The VA BAR **June 2026 - Aug 2026**

- Completed hands-on training in Administrative Support, Social Media Management, Email Marketing, and Lead Generation.
- Performed practical virtual assistant tasks using industry-standard tools and workflows.

SELECTED AUTOMATION PROJECTS

- InvoiceFlow AI - extracts invoice details and line items, detects duplicates, stores records, uploads files to Drive, and sends Slack alerts.
- SupportFlow AI - classifies support emails, creates ticket records, prepares Gmail draft replies, and notifies Slack
- HireFlow AI - parses resumes, scores candidates, maintains a Google Sheets ATS, and uses an ATS Interview Booking Tracker to update schedules automatically

EDUCATION

Bachelor of Science in Information Technology **Aug 2022 - May 2026**
Major in Web Development

Nueva Ecija University of Science and Technology

- Focused on Web Development, System Design, and Database Management Participated in group system development and capstone projects

Senior High School **June 2018 - June 2020**

Our Lady of Fatima University

- Developed foundational skills in communication, technology, and teamwork

CORE SKILLS

- Automation & AI: n8n, Webhooks, APIs, Groq, Google Gemini, Prompting
- Development: Next.js, React, TypeScript, JavaScript, Laravel, PHP, HTML, CSS
- Data & Deployment: Supabase, MySQL, Google Sheets, GitHub, Vercel, Railway
- Integrations: Gmail, Google Calendar, Google Drive, Slack, Shopify
- Business Support: Admin Support, Customer Support, CRM, Data EntryCreative & Office: Canva, Figma, Microsoft Office, Google Workspace

TECHNICAL PROPOSAL

AI AutomationSystemforCommercialPropertyManagement

Application Exercise
Backend / Automation Engineer (AI Systems) - Property management automation covering maintenance email triage, utility anomaly detection, and audit invoice package generation.

Prepared by	Michael Angelo L. Acuña
Profile	AI Automation Specialist | Virtual Assistant | Web Developer
Portfolio	https://real-portfolio-liard.vercel.app
Requested rate	$10 USD/hour, negotiable based on trial scope and weekly commitment
Availability	Part-time, approximately 20 hours/week, with up to 3 hours of weekday US Eastern overlap and additional asynchronous implementation time
Date	August 6, 2026

Relevant portfolio systems: SupportFlow AI, InvoiceFlow AI, HireFlow AI, LeadFlow AI

1. Executive Summary

Theproposed system is technically feasible as a phased, API-first automation platform, provided the client supplies appropriate Rent Manager andMicrosoft 365 access, representative lease and invoice samples, operational rules, and a designated review team. The system would centralize source data, automate repeatable processing, use AI only where judgment or document interpretation is needed, and route uncertain or high-risk decisions to human review.

The architecture separates deterministic business logic from AI-assisted interpretation. This makes each decision traceable, testable, and recoverable. The MVP would cover three independent but connected modules: maintenance email triage, utility cost anomaly detection, and audit invoice package generation. All modules would share a common datastore, workflow log, human-review queue, and monitoring layer.

Important scope note

Ihavenot yet implemented Rent Manager or Microsoft Graph in a live client environment. My approach is to validate their authentication, endpoint coverage, pagination, rate limits, and write-back capabilities during discovery before committing to final implementation details. My related experience includes email triage, invoice processing, Supabase data modeling, webhooks, structured AI outputs, file storage, and human-reviewed workflows.

2. Technical Feasibility and Constraints

2.1 Feasibility

Maintenance email triage is feasible through Microsoft Graph subscriptions or scheduled mailbox polling, with normalized email, attachment, sender, property, and tenant data stored before AI analysis.

Tenant-versus-landlord responsibility can be assisted by retrieving relevant lease clauses and applying explicit responsibility rules. Final decisions should remain reviewable, especially when the lease is unclear or the request has financial or legal impact.

Utility anomaly detection is primarily a deterministic analytics problem. Month-over-month, year-over-year, seasonal, occupancy-adjusted, and threshold-based comparisons should run before any AI-generated explanation.

Audit invoice packages can be generated by collecting approved invoices and supporting records, validating completeness, arranging files in a consistent order, and producing a downloadable package with an index and exception report.

2.2 Key constraints

Rent Manager API availability, licensing, permissions, sandbox access, endpoint limits, and write-back restrictions must be confirmed.

Microsoft 365 access may require tenant-admin approval for Graph scopes, webhook subscriptions, mailbox access, and application registration.

Lease language varies by property and jurisdiction. AI cannot replace legal review; the system should cite the exact clause used and escalate ambiguity.

Utility records may have inconsistent account numbers, billing cycles, meter changes, vacancies, credits, and estimated readings that can create false anomalies.

Historical data quality will determine the accuracy of property, tenant, lease, vendor, and invoice matching.

The client must define retention, privacy, access-control, and audit requirements before production deployment.

3. Proposed Architecture

Thesystem would use a modulararchitecture so each component can be tested, deployed, and improved independently. Supabase is proposed for theoperationaldatastore and review queue, while server-side TypeScript or Python workers handle API integrations, deterministic validation, and document processing. n8n can orchestrate scheduled and event-driven workflows where it improves visibility and maintainability.

Layer	Responsibilities	Candidate technologies
Source systems	Maintenance tickets, properties, tenants, utilities, invoices, vendors, work orders	Rent Manager, Microsoft 365, file storage
Ingestion	Authentication, pagination, webhook validation, normalization, deduplication	TypeScript/Python, n8n, Microsoft Graph

Layer	Responsibilities	Candidate technologies
Operational data	Canonical records, source snapshots, review items, workflow state, audit history	Supabase/PostgreSQL
Processing	Rules, anomaly calculations, document parsing, lease retrieval, AI classification	Server workers, LLM API, embeddings if needed
Human review	Approve, correct, reject, add notes, trigger write-back or package generation	Next.js review dashboard
Delivery	Notifications, exports, audit packages, approved write-backs	Email/Teams/Slack, PDF/ZIP, Rent Manager
Observability	Execution logs, failure alerts, retry queue, latency, reconciliation	workflow_runs, failed_jobs, monitoring alerts

3.1 High-level flow

Source event or scheduled sync is received and assigned a unique idempotency key.

Raw source payload is stored before transformation for traceability and safe replay.

Data is normalized into the canonical schema and matched to the correct organization, property, tenant, lease, vendor, or utility account.

Deterministic rules run first. AI is called only for classification, summarization, document interpretation, or explanation.

Schema validation, allowed-value checks, confidence thresholds, and source citations are applied to AI output.

High-confidence, low-risk items continue automatically; uncertain or high-impact items enter human review.

Approved actions are written back or packaged, and every state transition is added to the audit log.

4. Module Design

4.1 Maintenance email triage and responsibility decision

Ingest new maintenance emails and attachments through Microsoft Graph.

Extract sender, property indicators, unit, tenant, issue category, urgency, dates, and attachments.

Match the email to property, tenant, and active lease records using deterministic identifiers and fallback review rules.

Retrieve relevant lease clauses and property responsibility rules.

Produce a structured recommendation: issue category, urgency, likely responsible party, cited clause, confidence, missing information, and recommended next action.

Route emergency, low-confidence, disputed, or financially material cases to a reviewer before work-order creation or tenant communication.

4.2 Utility anomaly detection

Import normalized utility bills by property, meter/account, service type, billing period, usage, total cost, and rate components.

Compare month-over-month, same-month-prior-year, rolling average, cost-per-day, cost-per-unit, and occupancy-adjusted values.

Flag anomalies using configurable thresholds and robust statistics rather than relying on AI alone.

Use AI only to create a concise explanation from validated numbers and known context such as vacancies, leaks, rate changes, or estimated meter readings.

Require review before escalation when data is incomplete, an account changed, or the anomaly may be caused by credits or billing-cycle changes.

4.3 Audit invoice package generation

Accept an on-demand package request by property, vendor, date range, work order, or audit case.

Collect approved invoices, supporting emails, work orders, approvals, payment records, and exception notes.

Validate required documents, detect duplicates, and identify missing references before generation.

Create a package index, summary sheet, exception report, and consistently named PDF/ZIP output.

Store the generated package, checksum, version,requested-by user, andsourcerecord IDs for later reproduction.

5. Data Model Sketch

Everybusinesstableshouldincludeorganization_id, created_at, updated_at, source_system, source_external_id, and appropriateuniqueconstraints.Sensitive data access should be controlled by role and tenant.

Table	Representative fields	Purpose
organizations	id, name, timezone, settings	Multi-tenant boundary and client configuration
properties	id, organization_id, external_id, address, status	Canonical property record
units	id, property_id, external_id, unit_number	Unit-level matching
tenants	id, organization_id, external_id, contact data	Tenant identity and contact data
leases	id, property_id, tenant_id, start/end, document_url, status	Active lease and source document
lease_clauses	id, lease_id, clause_type, text, page, embedding	Retrievable, citable responsibility clauses
maintenance_requests	id, email_id, property_id, tenant_id, category, urgency, status	Normalized inbound maintenance case
responsibility_decisions	request_id, party, confidence, rule_result, cited_clause, reviewer	AI/rule recommendation and final human outcome
	id, property_id, account_id, period, usage, cost, source_file	Normalized utility history
	bill_id, metric, expected, actual, deviation, status	Detected anomalies and review result
utility_bills	id, property_id, vendor_id, invoice_no, date, amount, file_url, hash	Invoice record and duplicate protection
utility_anomalies	id, filters, status, output_url, checksum, version	
invoices		
audit_packages		Generated package and reproducibility metadata
human_reviews	id, entity_type, entity_id, reason, decision, reviewer, timestamps	Central review queue
workflow_runs	id, workflow, idempotency_key, state, attempts, started/completed	Execution history and reconciliation
failed_jobs	run_id, error_type, payload_ref, retry_after, resolved_at	Dead-letter and recovery queue

6. AI Reliability and Trustworthiness

AIshouldassistwith interpretation andlanguage,notsilentlycontrol high-risk business outcomes. The following safeguardswouldbe builtintothe workflow:

- Structured outputs validated against strict JSON schemas andenumerated values.
- Deterministic calculations and rules performed outside the model whenever possible.
- Retrieval of exact lease text with page or section citations included in the recommendation.
- Confidence thresholds tied to action type; low-confidence or high-impact items require review.
- Cross-checks between extracted values and source documents before database updates.
- Prompt and model version stored with each decision so results can be reproduced and compared.
- Evaluation fixtures containing representative emails, leases, bills, and invoices, including edge cases.
- Human correction captured as labeled feedback for rule and prompt improvement.
- No autonomous tenant, vendor, financial, or legal communication without explicit approved rules.

7. Reliability Strategy

Area	Implementation approach
Idempotency	Use source event IDs and unique business keys so retries cannot create duplicate requests, invoices, packages, or write-backs.
Retry handling	Retry only transient errors with exponential backoff and attempt limits; do not retry validation or permission failures blindly.
Dead-letter recovery	Persist failed payload references, error categories, and retry context in failed_jobs for review and safe replay.
Rate limits and pagination	Use checkpointed pagination, bounded concurrency, backoff, and provider-specific usage monitoring.
Reconciliation	Run scheduled comparisons between source-system counts/updated timestamps and local records to detect silent failures.
Monitoring	Alert on failed runs, stale subscriptions, unusual volume, queue age, missing expected records, and repeated low-confidence decisions.
Validation	Validate schemas, required foreign keys, totals, date ranges, file hashes, and allowed state transitions before writes.
Auditability	Store raw payload references, transformed data, model/rule outputs, approvals, write-backs, and user actions.
Security	Use least-privilege service accounts, server-side secrets, encryption in transit/at rest, role-based access, and regular credential rotation.

8. Project Phases, Effort, and Deliverables

Phase	Estimate	Primary deliverables
1. Discovery and access validation	3-5 business days	Confirm APIs, permissions, sample data, responsibility rules, package requirements, security constraints, and acceptance criteria.
2. Foundation and canonical data model	1-2 weeks	Set up environments, Supabase schema, source snapshots, workflow logs, error queues, authentication, and initial syncs.
3. Maintenance triage MVP	1-2 weeks	Email ingestion, property/tenant matching, lease retrieval, structured classification, review queue, and notifications.
4. Utility anomaly module	1 week	Historical normalization, anomaly rules, explanation layer, review workflow, and reporting.
5. Audit package module	1-2 weeks	Document collection, completeness checks, index/exception report, PDF/ZIP generation, and storage.
6. Hardening and deployment	1 week	Test fixtures, retries, reconciliation, monitoring, security review, documentation, training, and production launch.

Estimated MVP effort

Approximately180-240 hours over 6-8 weeks. The range depends mainly on Rent Manager API coverage, Microsoft 365 permissions, historical data quality, lease formats, write-back scope, and the number of required review and reporting interfaces.

9. Risk Management

APIaccess is delayedorincomplete: Start with mock fixtures and interface contracts; isolate adapters so production credentials canbeconnectedlater.

- **Source records do not match reliably:** Create explicit match-confidence states,exceptionqueues,and client-approved reconciliation rules.
- **Lease language is ambiguous:** Cite source clauses, expose uncertainty, and require human approval for disputed or material decisions.
- **Utility alerts create false positives:** Use configurable baselines, suppress known billing artifacts, and provide reviewer feedback loops.
- **Duplicate processing or write-back:** Enforce idempotency keys, unique constraints, transactional state changes, and write-back receipts.
- **Silent integration failure:** Use heartbeat checks, stale-subscription alerts, expected-volume monitoring, and scheduled reconciliation.
- **Scope expands before foundation is stable:** Use phase gates, acceptance criteria, change requests, and separate MVP from later enhancements.

10. Required Client Access and Dependencies

Rent Manager API documentation, sandbox credentials, allowed scopes, account identifiers, and representative source records.

Microsoft 365 test tenant or application registration with approved Graph permissions for required mailboxes and files.

Sample maintenance emails, attachments, leases, utility bills, invoices, work orders, and desired audit outputs.

Documented tenant-versus-landlord responsibility rules and designated reviewers for ambiguous cases.

Utility anomaly definitions, thresholds, known seasonal patterns, and escalation recipients.

Approved hosting, database, file storage, notification, and secrets-management environments.

Named technical and operational contacts who can answer source-system, process, and acceptance questions.

Privacy, retention, access-control, and audit requirements for tenant, lease, financial, and vendor data.

11. Optional Proof of Concept

Asmallproofofconceptcanbe deliveredbefore the full build using anonymized or mock data. It would demonstrate the proposedengineeringapproach withoutrequiring production Rent Manageraccess.

Receive a sample maintenance email through a webhook or test mailbox.

Normalize the message and match it to mock property, tenant, and lease records.

Retrieve a relevant lease clause and return a structured responsibility recommendation with citation and confidence.

Route low-confidence decisions to a simple Supabase review queue.

Record execution state, idempotency key, model output, reviewer decision, and failure details.

Asecondoptional spikecould packagesample invoicesand supportingdocumentsusingthesamevalidationand duplicate-detection concepts demonstratedin myInvoiceFlow AIportfolioproject.

12. Relevant Experience and Portfolio Evidence

SupportFlow AI: incoming email triage, category/priority/sentiment classification, ticket storage, human-reviewed Gmail drafts, and Slack notifications.

InvoiceFlow AI: PDF/image ingestion, structured invoice and line-item extraction, duplicate detection, Supabase storage, Google Drive files, confidence scoring, and team alerts.

HireFlow AI: resume parsing, candidate scoring, Google Sheets ATS records, and a connected interview-booking tracker that updates scheduled interview data.

LeadFlow AI: lead qualification, CRM storage, confirmation emails, Slack notifications, and a connected three-stage follow-up manager.

Portfolio: https://real-portfolio-liard.vercel.app

13. Closing

Myrecommended approach is to begin with a short paid discovery and integration spike, validate the Rent Manager and Microsoft 365interfaces, finalize the canonical data model, and then deliver one module at a time behind measurable acceptance criteria. This reduces integration risk, makes failures easier to diagnose, and gives the client usable value before the entire platform is complete.

I would be comfortable discussing this design, revising the assumptions based on actual API capabilities, and demonstrating how I build, test, and debug a smallerworkflowduringa paidtrial.